

实验进阶4 银行业务数据安全与AI应用

难度（最高三星）：★★★ 建议学时（2-4学时）：4学时

实验说明

学习地图

某银行是一家小型银行企业，有十多万客户。现在要为该银行开发一套银行业务系统，通过计算机对银行日常的业务进行处理，以保证数据的安全性，提高工作效率。

本系统的功能结构图如图1所示。

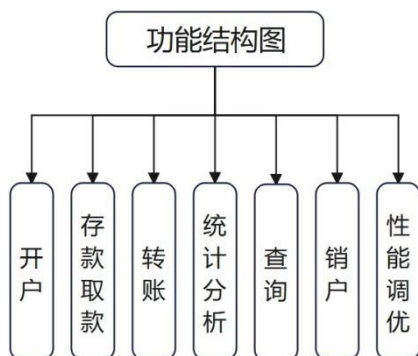


图1 银行业务系统功能结构图

本项目实验案例集要求根据银行业务需求设计出符合第3范式的数据库结构，使用SQL语言创建数据库和表，并添加表约束，进行数据的增、删、改、查，运用触发器、事务、视图和存储过程，按照银行的业务需求实现各项业务（包括日常存款、取款和转账等），同时为了适应日益增长的客户需求，要对数据库进行性能调优。

本项目将完成一个典型的数据库管理系统——银行业务系统的项目开发，要求完成的项目能够满足用户需求中的功能要求。共分为5个实验进阶、15个任务，按照难度等级分为初级、中级、高级，如图2所示。



图2 银行业务系统数据库

整个项目实验案例集将涉及openGauss数据库多个知识点和技能，如图3所示。

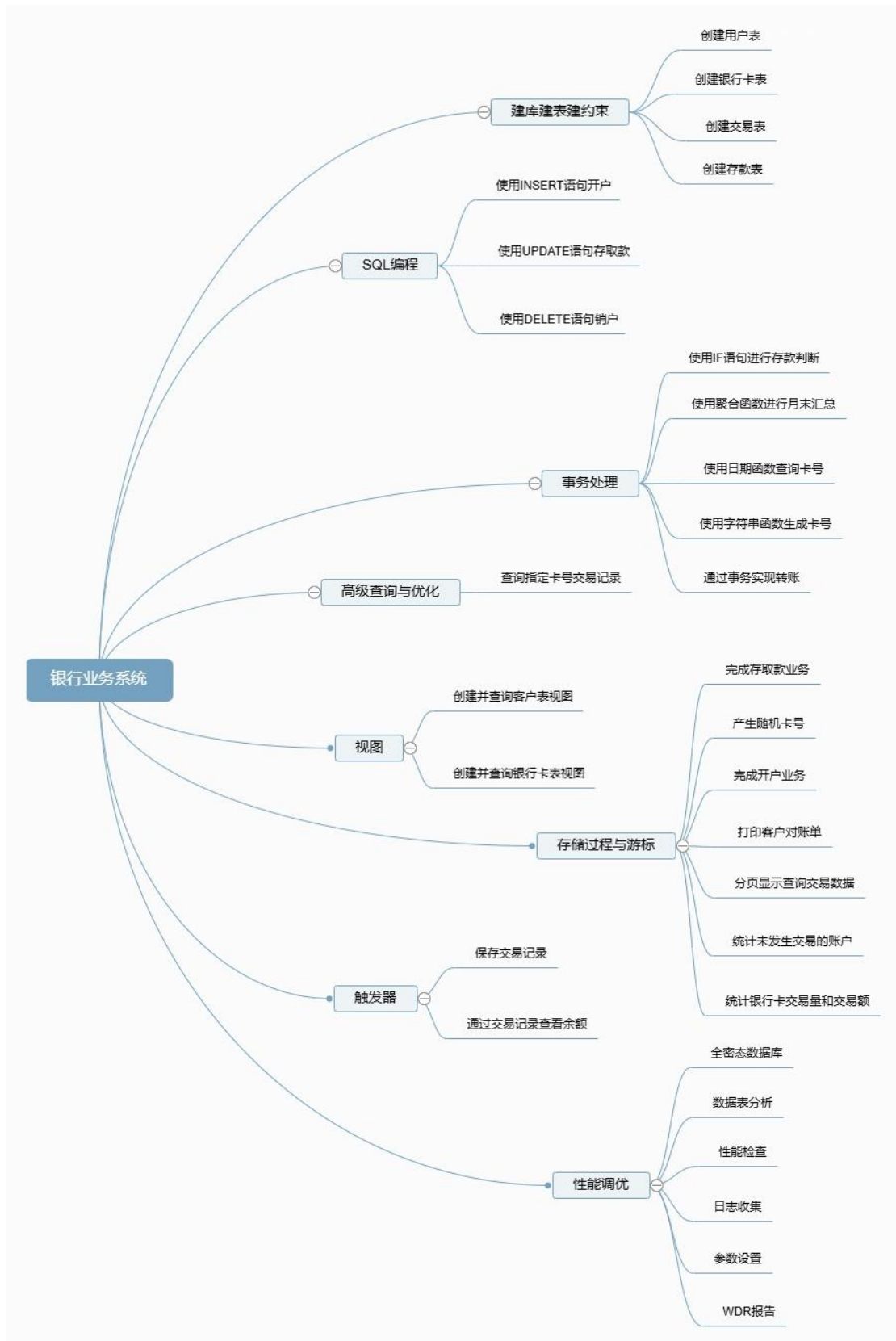


图3 银行业务系统知识点和技能

实验环境准备

具体开发环境如下：

- (1) 数据库：openGauss 5.0.2
- (2) 华为云
- (3) 终端工具：FinalShell，其安装和建立连接步骤见[网址](#)。

实验任务

任务描述

本次实验进阶4，主要是实现对银行业务数据系统的安全管理和人工智能应用，包括：

任务1：使用全密态数据库实现安全设计；

任务2：使用DB4AI实现房价预测。

学习目标

通过这两项任务能够快速掌握openGauss的数据加密和人工智能应用。

任务准备

前置知识

本实验案例，需提前学习openGauss的基本概念、作用及基础操作。可通过如下途径进行学习：

- 数字教材：《[数据库原理与应用：基于openGauss](#)》
- 技术文档：[openGauss白皮书](#)

任务实施

任务 1：使用全密态数据库实现安全设计

为了确保银行业务系统的数据安全和客户隐私安全，须采用全密态数据库和防篡改模式，保证数据在全生命周期中的加密状态，即使在数据传输、处理和存储环节遭遇攻击，也能确保攻击者无法获取任何有价值的信息。此外，防篡改功能通过记录和校验数据的操作历史，能有效防止数据被恶意篡改，确保了银行业务系统的完整性和可信度。

本实验将使用华为云来完成，OMM用是华为云用于系统管理的关键账户，主要用于集群管理和节点互信验证。

步骤1：在omm用户下创建\$GAUSSHOME/etc/localkms/路径。

```
mkdir -p $GAUSSHOME/etc/localkms/
```

步骤2：omm用户登录bankdb数据库，并开启密态开关。

```
gsql -d bankdb -p 26000 -r -C
```

步骤3：在bankdb下创建防篡改模式bank_security。

```
CREATE SCHEMA bank_security WITH BLOCKCHAIN;  
SET SEARCH_PATH TO bank_security;
```

步骤4：创建客户端主密钥CMK和列加密密钥CEK。

```
CREATE CLIENT MASTER KEY ImgCMK01 WITH (KEY_STORE = localkms, KEY_PATH =  
"key_path_value1", ALGORITHM = RSA_2048);  
CREATE CLIENT MASTER KEY ImgCMK02 WITH (KEY_STORE = localkms, KEY_PATH =  
"key_path_value2", ALGORITHM = RSA_2048);  
CREATE COLUMN ENCRYPTION KEY ImgCEK01 WITH VALUES (CLIENT_MASTER_KEY =  
ImgCMK01, ALGORITHM = AEAD_AES_256_CBC_HMAC_SHA256);  
CREATE COLUMN ENCRYPTION KEY ImgCEK02 WITH VALUES (CLIENT_MASTER_KEY =  
ImgCMK02, ALGORITHM = AEAD_AES_256_CBC_HMAC_SHA256);
```

步骤5：在bank_security模式下创建加密表。

```
CREATE TABLE userInfo
(
    customerID SERIAL PRIMARY KEY,
    customerName CHAR(8) NOT NULL,
    PID CHAR(18) ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = ImgCEK01,
    ENCRYPTION_TYPE = DETERMINISTIC) NOT NULL,
    telephone CHAR(20) NOT NULL,
    address VARCHAR(50)
);

CREATE TABLE cardInfo
(
    cardID CHAR(17) PRIMARY KEY,
    curID VARCHAR(10) DEFAULT 'RMB' NOT NULL,
    savingID INT NOT NULL DEFAULT 1,
    openDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    openMoney NUMERIC(18,2) NOT NULL,
    balance NUMERIC(18,2) ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = ImgCEK01,
    ENCRYPTION_TYPE = DETERMINISTIC) NOT NULL,
    pass CHAR(6) ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = ImgCEK02,
    ENCRYPTION_TYPE = DETERMINISTIC) NOT NULL DEFAULT '888888',
    IsReportLoss VARCHAR(3) NOT NULL DEFAULT '否',
    customerID INT NOT NULL
);

CREATE TABLE tradeInfo
(
    tradeDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    tradeType CHAR(6) NOT NULL,
    cardID CHAR(19) NOT NULL,
    tradeMoney NUMERIC(18,2) NOT NULL,
    remark TEXT
);
```

```
CREATE TABLE deposit
(
    savingID SERIAL PRIMARY KEY,
    savingName VARCHAR(20) NOT NULL,
    descrip VARCHAR(50)
);
```

步骤6：为表添加约束。因为密态数据库的特性，所以加密列不支持作为检查约束的一部分，对于加密列的约束可以在应用程序层面对明文进行验证，满足要求后再将其加密后存储到数据库中。

```
ALTER TABLE userInfo
    ADD CONSTRAINT chk_telephone CHECK
(CHAR_LENGTH(REGEXP_REPLACE(CAST(telephone AS TEXT), '[^0-9]', '', 'g')) = 11);

ALTER TABLE cardInfo
    ADD CONSTRAINT ck_cardid CHECK (
        SUBSTRING(cardID, 1, 8) = '10103576'
        AND SUBSTRING(cardID, 9) ~ '^[0-9]{8}$'
    ),
    ADD CONSTRAINT CK_openMoney CHECK (openMoney >= 1),
    ADD CONSTRAINT FK_savingID FOREIGN KEY(savingID) REFERENCES deposit(savingID);

ALTER TABLE tradeInfo
    ADD CONSTRAINT CK_tradeType CHECK(tradeType IN ('存入', '支取')),
    ADD CONSTRAINT CK_tradeMoney CHECK(tradeMoney > 0);
```

步骤7：插入数据

插入数据部分的代码可直接[下载相关代码](#)（如下）。

```
INSERT INTO deposit (savingName, descrip) VALUES ('Current Deposit', 'Interest is
calculated based on the deposit date');

INSERT INTO deposit (savingName, descrip) VALUES ('1Y Fixed', '1 year term');
INSERT INTO deposit (savingName, descrip) VALUES ('2Y Fixed', '2 year term');
INSERT INTO deposit (savingName, descrip) VALUES ('3Y Fixed', '3 year term');
```

```
INSERT INTO deposit (savingName) VALUES ('Flex Deposit');
INSERT INTO deposit (savingName, descrip) VALUES ('1Y Savings', '1 year term');
INSERT INTO deposit (savingName, descrip) VALUES ('2Y Savings', '2 year term');
INSERT INTO deposit (savingName, descrip) VALUES ('3Y Savings', '3 year term');
INSERT INTO deposit (savingName, descrip) VALUES ('5Y Int Only', 'monthly int');

INSERT INTO userInfo(customerid, customerName, PID, telephone, address)
VALUES(1, 'John', '11000000000000000001', '13900000001', 'Nanjing City, Jiangsu
Province');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(2,'Jane','11000000000000000002','13900000002');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(3,'Michael','11000000000000000003','13900000003');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(4,'Emily','11000000000000000004','13900000004');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(5,'David','11000000000000000005','13900000005');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(6,'Sarah','11000000000000000006','13900000006');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(7,'Robert','11000000000000000007','13900000007');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(8,'Jessica','11000000000000000008','13900000008');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(9,'William','11000000000000000009','13900000009');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(10,'Olivia','11000000000000000010','13900000010');

INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('10103576000000001', 1, 1000, 1000, 1);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('10103576000000002', 2, 1, 1, 2);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
```



```
VALUES ('1010357600000003', 2, 1, 1, 3);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000004', 2, 1, 1, 4);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000005', 3, 500, 500, 5);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000006', 3, 300, 300, 6);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000007', 3, 200, 200, 7);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000008', 3, 150, 150, 8);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000009', 3, 100, 100, 9);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000010', 3, 80, 80, 10);
```

查看userinfo表，可以看出在密态客户端查询时，数据仍以明文形式呈现，如图1.1所示。

```
SELECT * FROM userinfo;
```

customerid	customername	pid	telephone	address
1	John	110000000000000001	13900000001	江苏省南京市
2	Jane	110000000000000002	13900000002	
3	Michael	110000000000000003	13900000003	
4	Emily	110000000000000004	13900000004	
5	David	110000000000000005	13900000005	
6	Sarah	110000000000000006	13900000006	
7	Robert	110000000000000007	13900000007	
8	Jessica	110000000000000008	13900000008	
9	William	110000000000000009	13900000009	
10	Olivia	110000000000000010	13900000010	

图1.1 在密态客户端查看用户信息表

使用非密态客户端查询，可以看出在非密态客户端查看时，数据以密文的形式呈现，如图1.2所示。

```

gsqsl -d postgres -p 26000 -r
SET SEARCH_PATH TO bank_security;

SELECT * FROM userinfo;

SELECT * FROM cardinfo;

```

bankdb=# SELECT * FROM userinfo;					pid	add
customerid	customername	pass	telephone			

1	John	\x0185e07426d83ea311e0f6e02cb9338aaae7b1007175afa2608891a20f01586111119b39e310000007033e22	13900000001		江苏	南京市
2	Jane	\x0185e074264c7d37dc337d30bcb53dc0377db515c310a8242b1b9d52a235437e5a54a6610c310000003b5c209	13900000002			
3	Michael	\x0185e07426379a1880928a85648745c4294a2b70bb3481f85817533064d2bb563099c42fa31000000872b640	13900000003			
4	Emily	\x0185e0742680dc84833c00f3cecebd8c3fb31667bfebb5c878ea475aa77ff558607d37b6b6310000003faa92f	13900000004			
5	David	\x0185e074265b2102fbbd18da32bf9255deb0418477b3e7a92a720a463cc36dcfd443359ac3100000071ea27	13900000005			
6	Sarah	\x0185e074261c09fabb4e516fd695dcddff30ac85eb22d6a157a22b467ca509fe1dcf21b258310000007acc99	13900000006			
7	Robert	\x0185e07426489e38e40968351113f3c5e5bdf0198d7d54065194313f968a4fc242fa0cbdb6310000001787333	13900000007			
8	Jessica	\x0185e0742651649890697f6426dd331ea3654961f695ae3226a113bde4dd24f850cd8143310000004e0bb7a	13900000008			
9	William	\x0185e074260f7820e4739aac94bd082bcff9b1ae5c05d7d0f61ee5b93bdd41a75296488e4b310000005f11caf	13900000009			
10	Olivia	\x0185e074260605ca969e04ed442bc7a7ab0198758fd8f864974dbde73e7373ea643a9132cb3100000069f86e4	13900000010			
(10 rows)						

bankdb=# SELECT * FROM cardinfo;							
cardid	curid	savingid	opendate	openmoney	balance	pass	

isreportloss customerid							

1018357600000001	RMB	1	2024-08-09 16:44:35.983366	1000.00	\x0185e0742662e039faa05868dc75097a653f9	ebe505f82b906db3f8e5a215c2dabf128897a31000000db3b4ae17a8c481f70e4c66abc81be49f9a9ac21c2e5fad912da4b3b9077e0ae	\x01760f34db8e0b89806cc9977bc9d1607ee83fd588b49ca6b8c455ebf423b29b79d31bca1131000000e0f163d11c44b622a428147ea791197ff0e331409f2
1010357600000002	RMB	2	2024-08-09 16:44:35.992356	1.00	\x0185e0742675a8325e594ccbebb4b3e06a9f6	ff7b26380c2122b5a61344f5faf70e4dd039a31000000f11badf0ea886dac6a64def51808ebf2a4f613e3a4c2b88eb512d9c33aa4f3e6	\x01760f34db8e0b89806cc9977bc9d1607ee83fd588b49ca6b8c455ebf423b29b79d31bca1131000000e0f163d11c44b622a428147ea791197ff0e331409f2
1018357600000003	RMB	2	2024-08-09 16:44:36.001506	1.00	\x0185e0742675a8325e594ccbebb4b3e06a9f6	ff7b26380c2122b5a61344f5faf70e4dd039a31000000f11badf0ea886dac6a64def51808ebf2a4f613e3a4c2b88eb512d9c33aa4f3e6	\x01760f34db8e0b89806cc9977bc9d1607ee83fd588b49ca6b8c455ebf423b29b79d31bca1131000000e0f163d11c44b622a428147ea791197ff0e331409f2
1010357600000004	RMB	2	2024-08-09 16:44:36.010584	1.00	\x0185e0742675a8325e594ccbebb4b3e06a9f6	ff7b26380c2122b5a61344f5faf70e4dd039a31000000f11badf0ea886dac6a64def51808ebf2a4f613e3a4c2b88eb512d9c33aa4f3e6	\x01760f34db8e0b89806cc9977bc9d1607ee83fd588b49ca6b8c455ebf423b29b79d31bca1131000000e0f163d11c44b622a428147ea791197ff0e331409f2
1010357600000005	RMB	3	2024-08-09 16:44:36.019258	500.00	\x0185e07426a770730c5e606d7226fd6b84f494	0dbcb68906d145db6c6a49fcb548e36b35127231000000b579a1f174dea01f9c94169a6610f2d61dba5003bc9d04314b339d0f8828afc5	\x01760f34db8e0b89806cc9977bc9d1607ee83fd588b49ca6b8c455ebf423b29b79d31bca1131000000e0f163d11c44b622a428147ea791197ff0e331409f2
1018357600000006	RMB	3	2024-08-09 16:44:36.027699	300.00	\x0185e07426389812b9f11ce1e2b9cba3e83ca	b92228cb082cee0333a04f8b3b7b7664f015831000000467e7a7350d4b89ac9c168690cd71d65164ea0cfc17bcd4a7b542aeca048061b	\x01760f34db8e0b89806cc9977bc9d1607ee83fd588b49ca6b8c455ebf423b29b79d31bca1131000000e0f163d11c44b622a428147ea791197ff0e331409f2
1010357600000007	RMB	3	2024-08-09 16:44:36.036041	200.00	\x0185e074267d822ecf6dd8b3cfdcb76e77094		
--More--							

图1.2 在非静态客户端查看用户信息表

步骤8: 查看账本历史操作记录

查看执行DML的用户的执行时间和身份, 如图1.3所示。

```
SELECT * FROM gs_global_chain;
```

bankdb=# SELECT * FROM gs_global_chain;	blocknum	dbname	username	globalhash	txcommand	starttime	releid	relnsp	relname	relhash	
	1	bankdb	omm			2024-08-09 16:44:26.330344+08	16943	bank_security	deposit	fa409c4df29904606	d4
9be755c8bda5d61de245ffd276748					INSERT INTO deposit (savingName,descrip)	VALUES ('活期','按存款日结算利息');					
2	bankdb	omm				2024-08-09 16:44:26.331116+08	16943	bank_security	deposit	885167c815a6bf90	c9
387501ad2e64d008b8db4f6989af83					INSERT INTO deposit (savingName,descrip)	VALUES ('定期一年','存款期限1年');					
3	bankdb	omm				2024-08-09 16:44:26.331642+08	16943	bank_security	deposit	5fef36418c89d7b0	7e
242e71fad1fd4ba2e24b0639b41202					INSERT INTO deposit (savingName,descrip)	VALUES ('定期二年','存款期限2年');					
4	bankdb	omm				2024-08-09 16:44:26.332146+08	16943	bank_security	deposit	bee190ef5817766	20
aff77f1315e272447c371fb3c51281					INSERT INTO deposit (savingName,descrip)	VALUES ('定期三年','存款期限3年');					
5	bankdb	omm				2024-08-09 16:44:26.33258+08	16943	bank_security	deposit	e585e3fc3c06656b	35
c89c2e877052b3f2b8bd4fb484eb54					INSERT INTO deposit (savingName)	VALUES ('定活两便');					
6	bankdb	omm				2024-08-09 16:44:26.333089+08	16943	bank_security	deposit	fa74f6517a550610	11
c49d29e5660c9b46a3487aada4428c					INSERT INTO deposit (savingName,descrip)	VALUES ('零存整取一年','存款期限1年');					
7	bankdb	omm				2024-08-09 16:44:26.333601+08	16943	bank_security	deposit	403ba37aaa936523	dc
3fcec859c75bc7b92e8b12443e874f					INSERT INTO deposit (savingName,descrip)	VALUES ('零存整取二年','存款期限2年');					
8	bankdb	omm				2024-08-09 16:44:26.334129+08	16943	bank_security	deposit	1a906f6dda3af2c0	28
11dc9ad7d6c8e76efbc3519d4a3569					INSERT INTO deposit (savingName,descrip)	VALUES ('零存整取三年','存款期限3年');					
9	bankdb	omm				2024-08-09 16:44:27.239666+08	16943	bank_security	deposit	b5f9e85bf74f7fb1	42
1a936c18993b3ddeb7a81c806cf239					INSERT INTO deposit (savingName,descrip)	VALUES ('存本取息五年','按月支取利息');					
10	bankdb	omm				2024-08-09 16:44:35.984039+08	16911	bank_security	cardinfo	a2eae534fc2ed20	aa
4e418f653729034629feec6f8b8d5c					INSERT INTO cardInfo(cardID,savingID,openMoney,balance,customerID)						
					VALUES('1010357600000001',1,1000,'x0185e0742662e039faa05868dc75097a653f9ebe505f82						
b906db3f8e5a215c2dabf128897a31000000db3b4ae17a8c481f70e4c66abc81be49f9a9ac21c2e5fad912da4b3b9077e0ae',1);											
11	bankdb	omm				2024-08-09 16:44:35.992583+08	16911	bank_security	cardinfo	7b9a002d5f649fe8	dd
c40a7179dad9200be31db2859ec2df					INSERT INTO cardInfo(cardID,savingID,openMoney,balance,customerID)						
--More--											

图1.3 查看账本历史操作记录

任务 2：使用 DB4AI 实现房价预测

本实验需要安装DBMind，详细步骤将通过视频讲解。

DB4AI是指利用数据库的能力驱动AI任务，实现数据存储、技术栈的同构。通过在数据库内集成AI算法，令openGauss具备数据库原生AI计算引擎、模型管理、AI算子、AI原生执行计划的能力，为用户提供普惠AI技术。不同于传统的AI建模流程，DB4AI “一站式”建模可以解决数据在各平台的反复流转问题，同时简化开发流程，并可通过数据库规划出最优执行路径，让开发者更专注于具体业务和模型的调优上，具备同类产品不具备的易用性与性能优势。

openGauss支持原生DB4AI能力，通过引入原生AI算子，简化操作流程，充分利用数据库优化器、执行器的优化与执行能力，获得高性能的数据库内模型训练能力。更简化的

模型训练与预测流程、更高的性能表现，让开发者在更短时间内能更专注于模型的调优与数据分析上，而避免了碎片化的技术栈与冗余的代码实现。

模型训练语法如下。

创建模型。

```
CREATE MODEL
```

以multiclass为例，训练一个模型。

从tb_iris训练集中指定sepal_length, sepal_width, petal_length, petal_width为特征列，使用multiclass算法，创建并保存模型iris_classification_model。

```
CREATE MODEL iris_classification_model USING xgboost_regression_logistic FEATURES
sepal_length, sepal_width, petal_length, petal_width TARGET target_type < 2 FROM tb_iris_1
WITH nthread=4, max_depth=8;

MODEL CREATED. PROCESSED 1
```

查看模型信息：当训练完成后模型会被存储到系统表gs_model_warehouse中。系统表gs_model_warehouse可以查看到关于模型本身和训练过程的相关信息。

```
select * from gs_explain_model("model_name");
```

利用已存在的模型做推断任务：使用“SELECT”和“PREDICT BY”关键字利用已有模型完成推断任务。

```
SELECT...PREDICT BY...(FEATURES...)...FROM...;
```

查看执行计划：使用explain语句可对“CREATE MODEL”和“PREDICT BY”的模型训练或预测过程中的执行计划进行分析。Explain关键字后可直接拼接CREATE MODEL/PREDICT BY语句（子句），也可接可选的参数。

下面是模拟房价预测模型。

步骤1：创建house表，包括房屋 ID、税费、卧室数量、浴室数量、价格、面积和地块面积等，如图2.1所示。

```
CREATE TABLE houses (id INT, tax INT, bedroom INT, bath FLOAT, price INT, size INT, lot
INT);

INSERT INTO houses VALUES
(1, 590, 2, 1, 50000, 770, 22100),
(2, 1050, 3, 2, 85000, 1410, 12000),
(3, 20, 3, 1, 22500, 1060, 3500),
(4, 870, 2, 2, 90000, 1300, 17500),
(5, 1320, 3, 2, 133000, 1500, 30000),
(6, 1350, 2, 1, 90500, 820, 25700),
(7, 2790, 3, 2.5, 260000, 2130, 25000),
(8, 680, 2, 1, 142500, 1170, 22000),
(9, 1840, 3, 2, 160000, 1500, 19000),
(10, 3680, 4, 2, 240000, 2790, 20000),
(11, 1660, 3, 1, 87000, 1030, 17500),
(12, 1620, 3, 2, 118600, 1250, 20000),
(13, 3100, 3, 2, 140000, 1760, 38000),
(14, 2070, 2, 3, 148000, 1550, 14000),
(15, 650, 3, 1.5, 65000, 1450, 12000);
```

	id	tax	bedroom	bath	price	size	lot
1	1	590	2	1.0	50000	770	22100
2	2	1050	3	2.0	85000	1410	12000
3	3	20	3	1.0	22500	1060	3500
4	4	870	2	2.0	90000	1300	17500
5	5	1320	3	2.0	133000	1500	30000
6	6	1350	2	1.0	90500	820	25700
7	7	2790	3	2.5	260000	2130	25000
8	8	680	2	1.0	142500	1170	22000
9	9	1840	3	2.0	160000	1500	19000
10	10	3680	4	2.0	240000	2790	20000
11	11	1660	3	1.0	87000	1030	17500
12	12	1620	3	2.0	118600	1250	20000
13	13	3100	3	2.0	140000	1760	38000
14	14	2070	2	3.0	148000	1550	14000
15	15	650	3	1.5	65000	1450	12000

图2.1 查看创建后的house表

步骤2：通过CREATE MODEL语句，基于SVM算法创建一个二分类模型，通过房屋税、浴室数量和房屋面积预测房屋价格是否低于100000元。

```
CREATE MODEL house_binary_classifier USING svm_classification FEATURES tax, bath, size
TARGET price < 100000 FROM houses;
```

CREATE MODEL语句用于模型的训练和保存。

USING关键字指定算法名称。

FEATURES用于指定训练模型的特征，需根据训练数据表的列名添加。

TARGET指定模型的训练目标，它可以是训练所需数据表的列名，也可以是一个表达式，例如: price < 100000。

步骤3：在gs_model_warehouse系统表中查看训练后的模型信息，如图2.2所示，仅截取最后的部分。

```
SELECT * FROM gs_model_warehouse WHERE modelName = 'house_binary_classifier';
```

trainingscoresname	trainingscoresvalue	modeldescr...
{accuracy,f1,precision,recall,loss}	{.666666687,.666666687,.625,.714285731,7.10693932}	[NULL]

图2.2 在gs_model_warehouse系统表中查看训练后的模型信息

通过系统表的返回信息可以看到训练过程的预测准确率为 0.666667。

这对于二分类来说准确率并不高，我们后面将尝试通过超参数来提升模型表现。

步骤4：使用 DROP MODEL 语句删除模型。

```
DROP MODEL house_binary_classifier;
```

步骤5：使用更优的超参数提高SVM算法的训练表现，设置超参数:batch size 和学习率。

```
CREATE MODEL house_binary_classifier USING svm_classification FEATURES tax, bath, size
TARGET price < 100000 FROM houses WITH batch_size=1, learning_rate=0.001;
```

再次查看训练后的模型信息如图2.3所示。

trainingscoresname	trainingscoresvalue
{accuracy,f1,precision,recall,loss}	{.733333349,.714285731,.714285731,.714285731,9.30504322}

图2.3 查看训练后的模型信息

通过系统表的返回信息可以看到训练过程的预测准确率为 0.733333。

整体准确率要相对默认超参数有所提升。

步骤6：使用PREDICT BY 语句预测样本数据。

创建一个新的字段，表示价格是否小于 100000，保存预测值的信息，如图2.4所示。

```
SELECT tax, bath, size, price, price < 100000 AS price_actual, PREDICT BY
house_binary_classifier (FEATURES tax, bath, size) AS price_pred FROM houses;
```

	tax	bath	size	price	price_actual	price_pred
1	590	1.0	770	50000	☒	☒
2	1050	2.0	1410	85000	☒	☒
3	20	1.0	1060	22500	☒	☒
4	870	2.0	1300	90000	☒	☒
5	1320	2.0	1500	133000	☐	☒
6	1350	1.0	820	90500	☒	☐
7	2790	2.5	2130	260000	☐	☐
8	680	1.0	1170	142500	☐	☒
9	1840	2.0	1500	160000	☐	☐
10	3680	2.0	2790	240000	☐	☐
11	1660	1.0	1030	87000	☒	☐
12	1620	2.0	1250	118600	☐	☐
13	3100	2.0	1760	140000	☐	☐
14	2070	3.0	1550	148000	☐	☐
15	650	1.5	1450	65000	☒	☒

图2.4 创建一个新的字段保存预测值的信息

步骤7：通过 CREATE MODEL 语句创建一个逻辑回归（logistic regression）模型，通过房屋税、浴室数量和房屋面积预测房屋价格是否低于100000元。

```
CREATE MODEL house_logistic_classifier USING logistic_regression FEATURES tax, bath, size
target price < 100000 FROM houses WITH learning_rate=0.001;
```

步骤8：利用训练好的逻辑回归模型预测数据，并与SVM算法进行比较，如图2.5所示。

```
SELECT tax, bath, size, price, price < 100000 AS price_actual, PREDICT BY
house_binary_classifier (FEATURES tax, bath, size) AS price_svm_pred, PREDICT BY
house_logistic_classifier (FEATURES tax, bath, size) AS price_logistic_pred FROM houses;
```


	tax	bath	size	price	price_actual	price_svm...	price_logist...
1	590	1.0	770	50000	☒	☒	☒
2	1050	2.0	1410	85000	☒	☒	☒
3	20	1.0	1060	22500	☒	☒	☒
4	870	2.0	1300	90000	☒	☒	☒
5	1320	2.0	1500	133000	☐	☒	☒
6	1350	1.0	820	90500	☒	☐	☐
7	2790	2.5	2130	260000	☐	☐	☐
8	680	1.0	1170	142500	☐	☒	☒
9	1840	2.0	1500	160000	☐	☐	☐
10	3680	2.0	2790	240000	☐	☐	☐
11	1660	1.0	1030	87000	☒	☐	☐
12	1620	2.0	1250	118600	☐	☐	☐
13	3100	2.0	1760	140000	☐	☐	☐
14	2070	3.0	1550	148000	☐	☐	☐
15	650	1.5	1450	65000	☒	☒	☒

图2.5 训练好的逻辑回归模型预测数据与SVM算法模型比较结果

注意事项

- (1) 列加密密钥必须和主密钥——匹配。
- (2) 在DB4AI中，删除已创建的模型时，必须给当前用户赋予Drop权限。

随堂测试

1. 全密态数据库中业务数据流在数据处理过程中都是以密文形态存在，以下关于全密态数据库等值查询的描述，不正确的是（ ）
 - A. 全密态数据库有效保护数据在云上全生命周期的隐私安全，无论数据处于何种状态，攻击者都无法从数据库服务端获取有效信息。
 - B. 全密态数据库帮助云服务提供商获取第三方信任，用户通过将密钥掌握在自己手上，使得高权限用户无法获取数据有效信息。
 - C. 全密态数据库让云数据库借助全密态能力更好的遵守个人隐私保护方面的法律法规。
 - D. 由于数据是由密文形态存在，因此可以不需要防火墙进行保护。

2. 下列关于MOT的描述中，不正确的是（ ）

- A. MOT更适用于关键任务、性能敏感的在线事务处理
- B. MOT不能与基于磁盘的普通表并排创建
- C. MOT完全支持ACID特性
- D. MOT具备完全无锁的设计和高度的调优

3. （多选）以下哪几项能力属于openGauss数据库的DB4AI部分（ ）

- A. Predictor
- B. X-Tuner
- C. DeepSQL
- D. SQLdiag